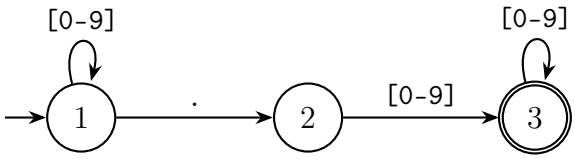
# PLT 2026 — Tutorial Week 3

**Generic Code** (used for all walkthroughs):

```
state = initial_state;
cur_char = getchar();
while (true) {
    if (cur_char == EOI)
        break;
    next_state = T[state][cur_char];
    if (next_state == ERROR)
        state = next_state;
        break;
    state = next_state;
    cur_char = getchar();
}
if (is_final_state(state))
    return (valid_token);
else
    return (not_valid_token);
```

## 1 Question 1: `[0-9]*.[0-9]+`

### 1.1 DFSA




State 1 = start (integer part); State 2 = dot read; State 3 = fractional part **(accept)**.

### 1.2 Transition Table

| **T** | `[0-9]` | `.` |
|---|---|---|
| → **1** | T(1) | T(2) |
| **2** | T(3) | T(error) |
| ***3** | T(3) | T(error) |

## 1.3 Walkthroughs

**Input:** `.75`

| Step | cur_char | T[state][cur_char] | state |
|---|---|---|---|
| init | — | — | 1 |
| 1 | . | T[1][.] = 2 | 2 |
| 2 | 7 | T[2][7] = 3 | 3 |
| 3 | 5 | T[3][5] = 3 | 3 |
| 4 | EOI | break | 3 |

`is_final_state(3)` = true ⇒ **valid_token**

**Input:** `86`

| Step | cur_char | T[state][cur_char] | state |
|---|---|---|---|
| init | — | — | 1 |
| 1 | 8 | T[1][8] = 1 | 1 |
| 2 | 6 | T[1][6] = 1 | 1 |
| 3 | EOI | break | 1 |

`is_final_state(1)` = false ⇒ **not_valid_token**

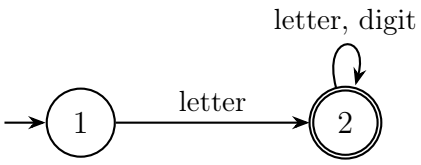
**Input:** `976.3`

| Step | cur_char | T[state][cur_char] | state |
|---|---|---|---|
| init | — | — | 1 |
| 1 | 9 | T[1][9] = 1 | 1 |
| 2 | 7 | T[1][7] = 1 | 1 |
| 3 | 6 | T[1][6] = 1 | 1 |
| 4 | . | T[1][.] = 2 | 2 |
| 5 | 3 | T[2][3] = 3 | 3 |
| 6 | EOI | break | 3 |

`is_final_state(3)` = true ⇒ **valid_token**

# 2 Question 2: `[a-zA-Z][a-zA-Z0-9]*`

## 2.1 DFSA




State 1 = start; State 2 = read at least one letter **(accept)**.

## 2.2 Transition Table

| **T** | **letter** | **digit** | **other** |
|---|---|---|---|
| → **1** | T(2) | T(error) | T(error) |
| ***2** | T(2) | T(2) | T(error) |

## 2.3 Walkthroughs

**Input:** `a92ad3`

| **Step** | **cur_char** | **T[state][cur_char]** | **state** |
|---|---|---|---|
| init | — | — | 1 |
| 1 | `a` (letter) | T[1][letter] = 2 | 2 |
| 2 | `9` (digit) | T[2][digit] = 2 | 2 |
| 3 | `2` (digit) | T[2][digit] = 2 | 2 |
| 4 | `a` (letter) | T[2][letter] = 2 | 2 |
| 5 | `d` (letter) | T[2][letter] = 2 | 2 |
| 6 | `3` (digit) | T[2][digit] = 2 | 2 |
| 7 | EOI | break | 2 |

`is_final_state(2)` = true ⇒ **valid_token**

**Input:** `3aty`

| **Step** | **cur_char** | **T[state][cur_char]** | **state** |
|---|---|---|---|
| init | — | — | 1 |
| 1 | `3` (digit) | T[1][digit] = ERROR<br>break | ERROR |

`is_final_state(ERROR)` = false ⇒ **not_valid_token**
(Starts with a digit, not a letter.)

**Input:** `ty5^`

| **Step** | **cur_char** | **T[state][cur_char]** | **state** |
|---|---|---|---|
| init | — | — | 1 |
| 1 | `t` (letter) | T[1][letter] = 2 | 2 |
| 2 | `y` (letter) | T[2][letter] = 2 | 2 |
| 3 | `5` (digit) | T[2][digit] = 2 | 2 |
| 4 | `^` (other) | T[2][other] = ERROR<br>break | ERROR |

`is_final_state(ERROR)` = false ⇒ **not_valid_token**
(`^` is not a letter or digit.)

# 3 Question 3: FLEX Regex for C Variable

**Rules:**

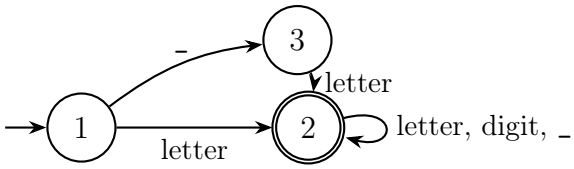
- Unlimited-length sequence of letters, digits, and underscores
- Must *not* begin with a digit
- If it begins with `_`, that must be followed by a letter, then unlimited letters/digits/underscores

**Examples:** `g`, `e17`, `_ty`, `r_u_ready`, `_t`.

## 3.1 Solution A — Alternation (`|`)

```
[a-zA-Z][a-zA-Z0-9_]*|_[a-zA-Z][a-zA-Z0-9_]*
```

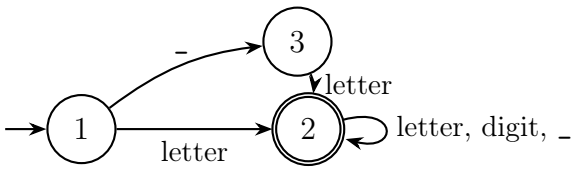
Two alternatives joined by `|`: the left branch starts with a letter, the right branch starts with `_` followed by a required letter. Both then allow unlimited continuation characters.




## 3.2 Solution B — Optional (`?`)

```
_?[a-zA-Z][a-zA-Z0-9_]*
```

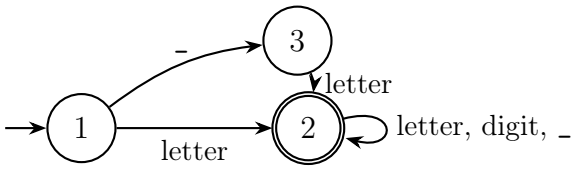
`_?` means zero or one underscore. After the optional underscore a letter is always required, then unlimited continuation. More concise than Solution A but describes the same language.




## 3.3 Solution C — Grouped alternation

```
([a-zA-Z]|_[a-zA-Z])[a-zA-Z0-9_]*
```

The opening group handles both start cases (plain letter *or* underscore-then-letter), then a single shared continuation `[a-zA-Z0-9_]*` follows.




**Note:** All three regular expressions describe the *same* language, so they produce the *same* minimal DFSA (3 states). The difference is purely syntactic—`|` (alternation), `?` (zero-or-one), and grouping `()` are three ways to express the optional leading underscore.

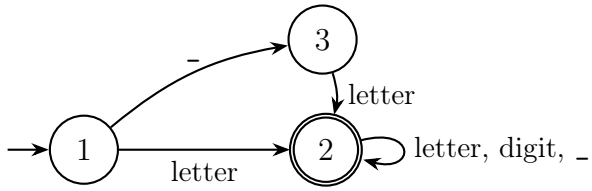
### 3.4 Verification Against Examples

| **String** | **A** | **B** | **C** |
|---|---|---|---|
| `g` | Yes | Yes | Yes |
| `e17` | Yes | Yes | Yes |
| `_ty` | Yes | Yes | Yes |
| `r_u_ready` | Yes | Yes | Yes |
| `_t` | Yes | Yes | Yes |
| `__x` (invalid) | No | No | No |
| `3ab` (invalid) | No | No | No |

# 4 Question 4: DFSA for C Variable Token

## 4.1 DFSA

(Same automaton for all three regex solutions above.)




State 1 = start; State 2 = valid identifier body **(accept)**; State 3 = read leading underscore, need a letter.

## 4.2 Transition Table

| T | letter | digit | _ | other |
|---|---|---|---|---|
| → **1** | T(2) | T(error) | T(3) | T(error) |
| ***2** | T(2) | T(2) | T(2) | T(error) |
| **3** | T(2) | T(error) | T(error) | T(error) |

# 5 Question 5: Walkthroughs for C Variable Token

**Input:** `Ay65__`

| Step | cur_char | T[state][cur_char] | state |
|---|---|---|---|
| init | — | — | 1 |
| 1 | `A` (letter) | T[1][letter] = 2 | 2 |
| 2 | `y` (letter) | T[2][letter] = 2 | 2 |
| 3 | `6` (digit) | T[2][digit] = 2 | 2 |
| 4 | `5` (digit) | T[2][digit] = 2 | 2 |
| 5 | `_` (underscore) | T[2][_] = 2 | 2 |
| 6 | `_` (underscore) | T[2][_] = 2 | 2 |
| 7 | EOI | break | 2 |

`is_final_state(2)` = true ⇒ **valid_token**

**Input:** `H67$%`

| Step | cur_char | T[state][cur_char] | state |
|---|---|---|---|
| init | — | — | 1 |
| 1 | `H` (letter) | T[1][letter] = 2 | 2 |
| 2 | `6` (digit) | T[2][digit] = 2 | 2 |
| 3 | `7` (digit) | T[2][digit] = 2 | 2 |
| 4 | `$` (other) | T[2][other] = ERROR<br>break | ERROR |

`is_final_state(ERROR)` = false ⇒ **not_valid_token**
(`$` is not a letter, digit, or underscore.)